

Assignment 2: Building Resilient Distributed Systems

Muhammad Talha Jamil | bsai23008 | Parallel and Distributed Computing

Part 1: Analysis of the Broken Architecture

Problem 1 - Lost Update (Synchronization)

The root cause here is the absence of any concurrency control at the database layer. When User A and User B both open the same shared document, they each receive an identical snapshot of the current state. Both then make independent edits and send a PUT/PATCH request to the FastAPI backend. The backend performs a simple read-then-write without checking whether the record changed between the two reads. Whichever request arrives last wins, and the other user's changes are silently discarded. This is the classic Lost Update anomaly and it happens because the API treats each write as if it owns an exclusive lock on the row, which it does not. In a naive SQLAlchemy setup, a plain `session.query().filter().update()` has no awareness of concurrent writers, so there is nothing stopping two transactions from clobbering each other.

Problem 2 - Dropped Webhook (Coordination)

Clerk sends a one-shot HTTP POST to the application's webhook endpoint when a subscription is cancelled. The current architecture processes this event synchronously inside the request handler and then returns a 200 OK. If a network blip, server restart, or transient database error happens during that window, the handler either never receives the request or crashes before committing the state change. Clerk does retry failed webhooks, but if the endpoint returns 200 before finishing the actual DB write (or the server crashes mid-write), Clerk considers the delivery successful and never retries. The result is permanent state drift: Clerk records the user as free-tier, but our database still shows them as premium. There is no idempotency check, no retry queue, and no dead-letter mechanism to catch these failures.

Problem 3 - Blocking LLM Call (Fault Tolerance)

The FastAPI route that calls the external LLM API does so synchronously with no timeout and no fallback. FastAPI is an async framework, but if the developer used the requests library or a blocking `httpx` call inside an async route without `await`, the entire event loop thread blocks for the full duration of the LLM timeout, which can be 60 seconds or more. Even with proper async usage, holding a connection open for 60 seconds per user means the server exhausts its connection pool quickly under modest load. There is no circuit breaker to detect that the LLM is unhealthy and stop sending requests to it, so every user during that outage period sits waiting until their request times out as well. This makes a third-party API failure a single point of failure for the entire application.

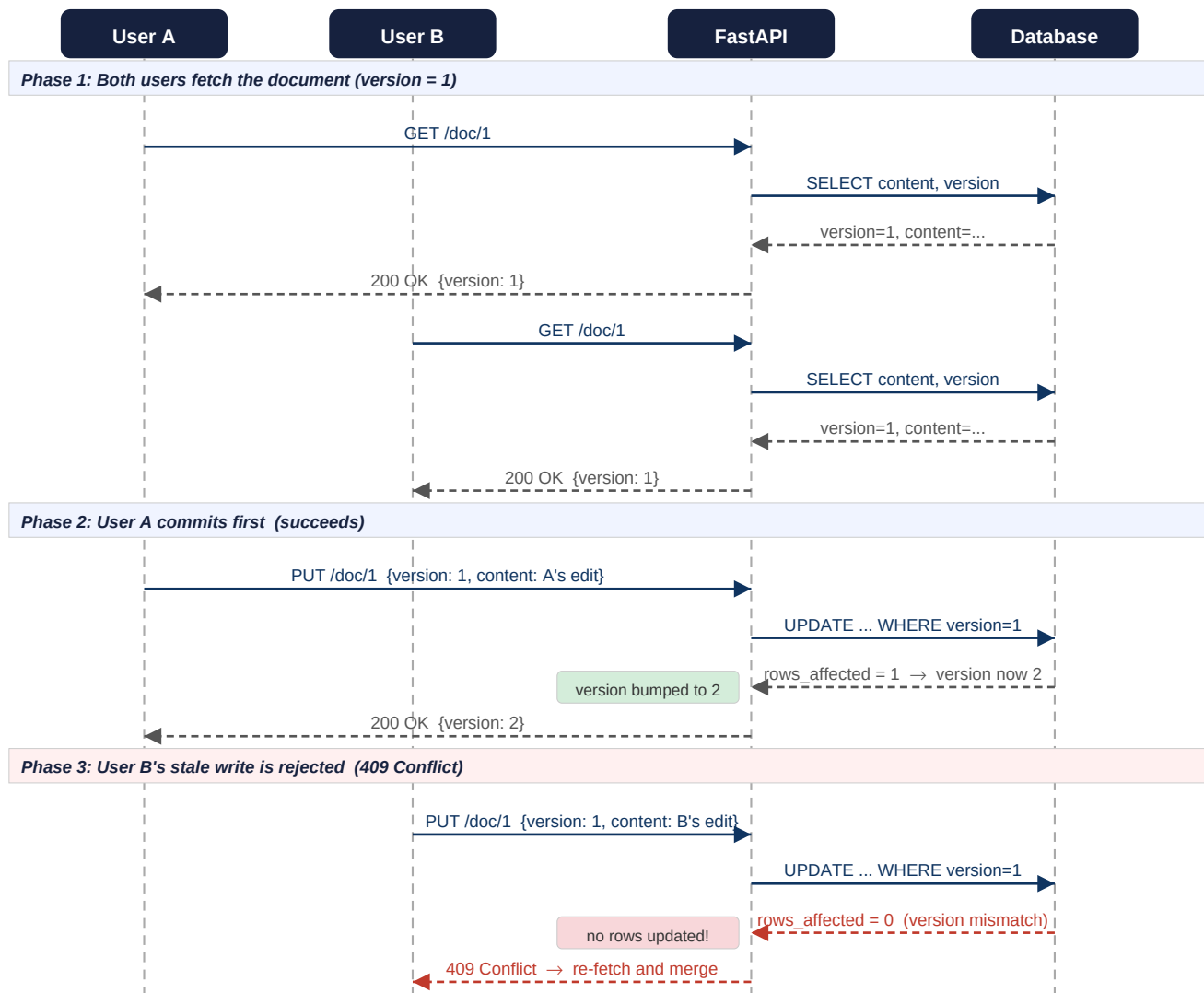
Part 2: Architectural Solutions

Fix 1 - Optimistic Locking for Concurrent Edits

The fix is Optimistic Locking using a version counter on the documents table. Every row gets a *version* integer column that starts at 1 and increments on every successful write. When a client fetches a document it also receives the current version number. When it submits an edit, it sends that version back in the request body.

The backend then executes a conditional update: `UPDATE documents SET content=:new, version=version+1 WHERE id=:id AND version=:expected`. If another writer already incremented the version, this `WHERE` clause matches zero rows. The backend detects this (`rows_affected == 0`) and returns HTTP 409 Conflict. The client can then re-fetch the latest version, show the user a merge prompt, and retry. No changes are silently lost. This approach has very low overhead because it adds only one integer column and one condition to an existing `UPDATE` query.

UML Sequence Diagram: Concurrent Edit with Optimistic Locking



Fix 2 - Fault-Tolerant Webhook Handling

The webhook endpoint should do two things immediately on receipt: validate the Clerk signature and persist the raw event to a `webhook_events` table with a status of 'pending', using the event ID provided by Clerk as a unique idempotency key. It then returns 200 right away so Clerk does not retry unnecessarily. A background worker (a simple `asyncio` task or a Celery worker) then picks up pending events and processes them. If processing fails, the worker retries with exponential backoff. After a configurable number of retries the event is moved to a dead-letter queue (a separate DLQ table or topic) and an alert is fired. Because the event ID is the

idempotency key, even if Clerk retries the same webhook the handler will detect the duplicate and skip reprocessing. This guarantees at-least-once delivery and prevents both drops and double-processing.

Fix 3 - Circuit Breaker with Fallback for LLM Calls

A Circuit Breaker sits in front of every call to the LLM API. It tracks consecutive failures and has three states. In the CLOSED state, requests pass through normally. When failures exceed a threshold (e.g., 5 in a row), it flips to OPEN and immediately returns a fallback response without hitting the LLM at all, so users get a fast degraded response instead of a 60-second hang. After a cooldown period (e.g., 30 seconds), it enters HALF-OPEN and allows one probe request through. If that succeeds the breaker resets to CLOSED; if it fails it goes back to OPEN. The fallback response can be a cached answer, a message like 'AI features are temporarily unavailable, please try again shortly', or a simpler non-AI code path. This completely decouples our application availability from the LLM provider's uptime.

CAP Theorem Trade-offs

Problem	CAP Choice	What we gain / what we give up
Optimistic Locking	CP (Consistency + Partition Tolerance)	We never silently lose a write. If two users clash, one gets a 409 and must retry. The data is always correct. The cost is that concurrent writers are temporarily blocked from succeeding — reduced write availability.
Webhook Queue	AP (Availability + Partition Tolerance)	We return 200 immediately so Clerk thinks delivery succeeded. The actual DB update happens asynchronously. During a partition the event sits in the queue — Clerk and our DB may be briefly out of sync, but no event is ever lost.
Circuit Breaker	AP (Availability + Partition Tolerance)	When the LLM is down we return a degraded fallback instantly instead of hanging for 60 seconds. The app stays available. The trade-off is that the response is not a real AI answer — consistency with the LLM's output is sacrificed temporarily.

In summary, there is no single right answer in CAP trade-offs. The optimistic locking fix prioritizes correctness (CP), the webhook fix leans AP to avoid dropping critical business events, and the circuit breaker also leans AP to keep the app responsive. The key insight is that different parts of the same system can make different CAP choices depending on what the cost of inconsistency is for that specific feature.